

Concurrent Uebungen

Euclid

$$|\mathbf{x}| = \sqrt{\sum_{k=1}^n |x_k|^2}$$

Erstelle ein **CompletableFuture**, das die **euklidische Norm** für eine Liste von **Number** berechnet.

Beispiel:

Beispiel: Gegeben ist folgende Collection: [1, 2, 3, 4, 5.5]

Berechnung: Wurzel(1*1 + 2*2 + 3*3 + 4*4 + 5.5*5.5) // *ergibt 7.762087348130012*

Tipp: Die [Math](#)-Bibliothek könnte hilfreich sein.

Das **Ergebnis** soll erst **nach zwei Sekunden zur Verfügung** stehen (Thread::sleep).

Während das CompletableFuture rechnet, sollen auf der Konsole **alle 100 ms ein Punkt "." ausgegeben** werden, bis die Berechnung des Futures fertig ist. Gebe dann das **Ergebnis des CompletableFuture auf der Konsole** aus.

Euclid: Lösung

```
class EuclideanNormCalculator {  
  
    public static void main(String[] args) {  
        CompletableFuture<Double> euclideanNormCalculation = CompletableFuture.supplyAsync(() -> {  
            sleep(Duration.ofSeconds(2));  
            Collection<Number> numbers = List.of(1, 2, 3, 4, 5.5);  
            return euclideanNorm(numbers);  
        });  
  
        while (!euclideanNormCalculation.isDone()) {  
            System.out.println(".");  
            sleep(Duration.ofMillis(100));  
        }  
  
        double result = euclideanNormCalculation.join();  
        System.out.println(result);  
    }  
  
    private static void sleep(Duration duration) {  
        try {  
            Thread.sleep(duration.toMillis());  
        } catch (InterruptedException e) {  
            throw new RuntimeException(e);  
        }  
    }  
  
    private static double euclideanNorm(Collection<Number> numbers) {  
        double sum = numbers.stream()  
            .mapToDouble(number -> number.doubleValue() * number.doubleValue())  
            .sum();  
        return Math.sqrt(sum);  
    }  
}
```

ConcurrentCurrencyConverter

```
class ConcurrentCurrencyConverter {  
  
    private static final Random RANDOM = new Random();  
  
    private static double getExchangeRate(String currency) { // Simuliert einen API-Call für den Wechselkurs  
        try {  
            int millis = RANDOM.nextInt(2000) + 500;  
            Thread.sleep(millis);  
        } catch (InterruptedException e) {  
            Thread.currentThread().interrupt();  
        }  
  
        return switch (currency) { // Simulierte Wechselkurse  
            case "USD" -> 1.08;  
            case "GBP" -> 0.85;  
            case "JPY" -> 158.0;  
            default -> throw new IllegalArgumentException("Unbekannte Währung");  
        };  
    }  
  
    public static void convertAmount(double euroAmount) {  
        // TODO: Implementiere hier die parallele Umrechnung mit CompletableFuture.  
        // Wandle den Betrag 42 Euro parallel in USD, GBP und JPY um.  
        // Gib jedes Ergebnis in der Konsole aus, sobald es verfügbar ist inklusive der Berechnungszeit.  
        // Zeige am Ende die Gesamtzeit an.  
    }  
  
    public static void main(String[] args) {  
        convertAmount(42);  
    }  
}
```

Mögliche Ausgabe:

```
GBP: 42,00 EUR = 35,70 GBP (0,6 s)  
USD: 42,00 EUR = 45,36 USD (1,0 s)  
JPY: 42,00 EUR = 6636,00 JPY (1,2 s)  
Alle Umrechnungen abgeschlossen (1,2 s)
```

ConcurrentCurrencyConverter (Lösung)

```
import java.time.Duration;
import java.time.Instant;
import java.util.Random;
import java.util.concurrent.CompletableFuture;

class ConcurrentCurrencyConverter {

    private static final Random RANDOM = new Random();

    // Simuliert einen API-Call für den Wechselkurs
    private static double getExchangeRate(String currency) {
        try {
            int millis = RANDOM.nextInt(2000) + 500;
            Thread.sleep(millis);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }

        // Simulierte Wechselkurse
        return switch (currency) {
            case "USD" -> 1.08;
            case "GBP" -> 0.85;
            case "JPY" -> 158.0;
            default -> throw new IllegalArgumentException("Unbekannte Währung");
        };
    }

    public static void convertAmount(double euroAmount) {
        Instant start = Instant.now();

        CompletableFuture<Void> usdFuture = new CompletableFuture(euroAmount, "USD", start);
        CompletableFuture<Void> gbpFuture = new CompletableFuture(euroAmount, "GBP", start);
        CompletableFuture<Void> jpyFuture = new CompletableFuture(euroAmount, "JPY", start);

        // Warte bis alle Umrechnungen abgeschlossen sind.
        CompletableFuture.allOf(usdFuture, gbpFuture, jpyFuture).join();

        Duration totalTime = Duration.between(start, Instant.now());
        System.out.printf("Alle Umrechnungen abgeschlossen (%.1f s)", totalTime.toMillis() / 1000.0);
    }

    private static CompletableFuture<Void> newCompletableFuture(double euroAmount, String targetCurrency, Instant start) {
        return CompletableFuture
            .supplyAsync(() -> getExchangeRate(targetCurrency))
            .thenAccept(rate -> printConversion(euroAmount, rate, targetCurrency, start));
    }

    private static void printConversion(double euroAmount, double exchangeRate, String targetCurrency, Instant start) {
        double convertedAmount = euroAmount * exchangeRate;
        Duration duration = Duration.between(start, Instant.now());
        System.out.printf("%s: %.2f EUR = %.2f %s (%.1f s)%n",
            targetCurrency, euroAmount, convertedAmount, targetCurrency, duration.toMillis() / 1000.0);
    }

    public static void main(String[] args) {
        convertAmount(42);
    }
}
```

Parallel Stream

1. Erstelle eine Liste mit folgenden Namen:
`"Paul", "Vladimir", "Jessica", "Duncan", "Chani", "Leto"`
2. Implementiere Folgendes einmal mit einem seriellen und einmal mit einem parallelen Stream:
 1. Es sollen nur Namen bearbeitet werden, deren **Buchstabenanzahl größer gleich 5** ist.
 2. Wandle diese Namen in **Großbuchstaben** um.
 3. **Schlafe** vor jeder Umwandlung für **100 ms**.
 4. Gebe alle umgewandelten Namen schließlich auf der Konsole aus.
3. Berechne die Laufzeit mit **java.time.Instant** für beide Varianten.
Gebe die Dauer für beide Varianten in Millisekunden auf der Konsole aus.

Ausgabe:

```
Serial:    [VLADIMIR, JESSICA, DUNCAN, CHANI]
Duration: 438 ms
Parallel:  [VLADIMIR, JESSICA, DUNCAN, CHANI]
Duration: 106 ms
```

Parallel Stream (Lösung)

```
class ParallelStreamExample {  
  
    public static void main(String[] args) {  
        List<String> names = List.of("Paul", "Vladimir", "Jessica", "Duncan", "Chani", "Leto");  
  
        Instant start = Instant.now();  
        List<String> resultSerial = newTransformStream(names).toList();  
        Instant end = Instant.now();  
        printResults("Serial: ", resultSerial, start, end);  
  
        start = Instant.now();  
        List<String> resultParallel = newTransformStream(names).parallel().toList();  
        end = Instant.now();  
        printResults("Parallel: ", resultParallel, start, end);  
    }  
  
    private static Stream<String> newTransformStream(List<String> names) {  
        return names.stream()  
            .filter(name -> name.length() >= 5)  
            .map(name -> {  
                sleep(Duration.ofMillis(100));  
                return name.toUpperCase();  
            });  
    }  
  
    private static void sleep(Duration duration) {  
        try {  
            Thread.sleep(duration.toMillis());  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
    }  
  
    private static void printResults(String method, List<String> result, Instant start, Instant end) {  
        System.out.println(method + result);  
        System.out.println("Duration: " + Duration.between(start, end).toMillis() + " ms");  
    }  
}
```

DRY:

Mit der Intermediate-Methode `parallel()` kann man einen vorhandenen seriellen Stream in einen parallelen Stream umwandeln.

Beachte: Die Methode `newTransformStream` gibt immer einen neuen Stream zurück, weil ein und derselbe Stream nicht wiederverwendet werden kann. Sobald die terminale Methode `toList()` aufgerufen wird, ist der Stream nicht mehr verwendbar.

Reduce

Berechne **10 + die Summe aus folgenden Zahlen**: [10, 20, 30, 40]. Verwende hierzu die Methode **Stream::reduce**.

Schreibe eine Lösung sowohl mit **Stream** als auch mit **Parallel-Stream**.

Gebe beide Ergebnisse in der Konsole aus.

Erwartetes Ergebnis:

110

Reduce: Lösung

```
class Reduce {  
  
    static int reduceSerial(int startValue, Collection<Integer> numbers) {  
        return numbers.stream().reduce(startValue, (a, b) -> a + b);  
        // return numbers.stream().reduce(startValue, Integer::sum);  
    }  
  
    static int reduceParallel(int startValue, Collection<Integer> numbers) {  
        return numbers.parallelStream().reduce(startValue, (a, b) -> a + b);  
        // return numbers.stream().reduce(startValue, Integer::sum);  
    }  
  
    public static void main(String[] args) {  
        var numbers = List.of(10, 20, 30, 40);  
  
        System.out.println(reduceSerial(10, numbers)); // 110  
        System.out.println(reduceParallel(10, numbers)); // 140  
    }  
}
```

Solange du nur pure Lambdas verwendest, lassen sich alle Streams in Parallel-Stream umwandeln.

Ausnahme: reduce mit Startwert. Hier wird für jedes Element in numbers der Startwert +10 hinzuaddiert. Das ist zugegeben nicht gerade intuitiv...

JokeFetcher: StringParser

Lade das Projekt "**JokeFetcher**" aus **Moodle** herunter.

JokeFetcher ruft zufällige Jokes aus folgender API ab (siehe [Doku](https://official-joke-api.appspot.com/jokes/random)):
<https://official-joke-api.appspot.com/jokes/random>

Die Antworten liegen im **JSON-Format** vor. Beispiel:

```
{"type":"programming","setup":"A SQL query walks into a bar, walks up to  
two tables and asks...","punchline":"'Can I join you?',"id":24}
```

Implementiere zunächst die Klasse **JokeJsonStringParser**, sodass der zur Verfügung gestellte JUnit-Test **JokeJsonStringParserTest** deine Lösung akzeptiert.

Du darfst hierfür **nur Methoden aus der Klasse String verwenden**. Schaue in der Klasse [String](#) nach, welche Methode nützlich sein könnten und wie du die Methode **split** verwenden kannst.

Gehe bei deiner Implementierung davon aus, dass die Struktur des JSON immer gleich ist, d. h., es gibt immer ein type, ein setup, eine punchline, eine id und diese sind immer in gleicher Reihenfolge und es gibt immer Werte (kein null).

JokeFetcher: StringParser (Lösung)

Anmerkung: Unterschiedliche Lösungswege sind hier möglich.

```
class JokeJsonStringParser implements JokeJsonParser {
```

```
    @Override
```

```
    public Joke parse(String json) {
        var attributes = parseKeyValues(json);
        return new Joke(
            Integer.parseInt(attributes.get("id")),
            attributes.get("type"),
            attributes.get("setup"),
            attributes.get("punchline")
        );
    }
}
```

```
private static Map<String, String> parseKeyValues(String json) {
```

```
    Map<String, String> result = new HashMap<>(); // key, value
    var cleanedJson = removeUnnecessaryCharacters(json);
    var keyValuePairs = cleanedJson.split(",");
    for (String pair : keyValuePairs) {
        var keyValue = pair.split(":");
        result.put(keyValue[0], keyValue[1]);
    }
    return result;
}
```

```
private static String removeUnnecessaryCharacters(String json) {
```

```
    return json
        .replace("\\\"", "\"")
        .replace("{", "")
        .replace("}", "");
}
```

```
class JokeJsonStringParser implements JokeJsonParser {
```

```
    private static final String ID = "\"id\"";
    private static final String TYPE = "\"type\"";
    private static final String SETUP = "\"setup\"";
    private static final String PUNCHLINE = "\"punchline\"";
```

```
    @Override
```

```
    public Joke parse(String json) {
        return new Joke(getId(json), getType(json), getSetup(json), getPunchline(json));
    }
```

```
    private int getId(String json) {
        int startIdx = json.indexOf(ID) + ID.length();
        int endIdx = json.lastIndexOf("}");
        return Integer.parseInt(json.substring(startIdx, endIdx));
    }
```

```
    private String getType(String json) {
        return parseFromTo(json, TYPE, SETUP);
    }
```

```
    private String getSetup(String json) {
        return parseFromTo(json, SETUP, PUNCHLINE);
    }
```

```
    private String getPunchline(String json) {
        return parseFromTo(json, PUNCHLINE, ID);
    }
```

```
    private String parseFromTo(String json, String start, String end) {
        int startIdx = json.indexOf(start) + start.length();
        final var endIdxOffset = -2;
        int endIdx = json.indexOf(end) + endIdxOffset;
        return json.substring(startIdx, endIdx);
    }
}
```

JokeFetcher: JacksonParser

Implementiere nun die Klasse **JokeJsonJacksonParser**, sodass der zur Verfügung gestellte JUnit-Test **JokeJsonJacksonParserTest** deine Lösung akzeptiert.

Nutze für diese Implementierung **Jackson**. Jackson ist eine weit verbreitete Java-Bibliothek, die zum Parsen von JSON verwendet wird. Die Dependency ist bereits in dem zur Verfügung gestellten Projekt enthalten.

Recherchiere im Internet, wie du Jackson verwenden kannst.

JokeFetcher: JacksonParser (Lösung)

```
class JokeJsonJacksonParser implements JokeJsonParser {  
  
    @Override  
    public Joke parse(String json) {  
        try {  
            return new ObjectMapper().readValue(json, Joke.class);  
        } catch (JsonProcessingException e) {  
            throw new RuntimeException(e);  
        }  
    }  
}
```

JokeFetcher: fetchRandomJokes

Implementiere schließlich die folgende Methode in der Klasse **JokeFetcher**:

```
/**
 * Requests a certain number of random jokes from a joke API.
 * All requests are made <b>asynchronous (in parallel)</b>.
 * The jokes are parsed from JSON to {@link Joke}.
 *
 * @param limit a positive number of jokes to be fetched.
 * @return a collection with fetched and parsed {@link Joke} objects.
 * @throws IllegalArgumentException if limit is below zero.
 * @see HttpClient#sendAsync
 * @see CompletableFuture
 */
Collection<Joke> fetchRandomJokes(int limit) {
    return new LinkedList<>(); // TODO
}
```

Die Methode soll eine bestimmte Anzahl (limit) an zufälligen Jokes aus der genannten Joke-API abrufen. Dies muss **parallel implementiert** werden: mehrere API-Anfragen sollen **gleichzeitig** gestellt werden. Die Ergebnisse müssen **geparst** werden (siehe **JokeJsonParser**) und werden als Collection zurückgegeben.

Prüfe deine Lösung mit dem zur Verfügung gestellten JUnit-Test **JokeFetcherTest**.

JokeFetcher: fetchRandomJokes (Lösung)

```
class JokeFetcher {
    private static final String JOKE_API_URL = "https://official-joke-api.appspot.com/jokes/random";
    private final HttpClient httpClient;
    private final JokeJsonParser jokeJsonParser;

    JokeFetcher(JokeJsonParser jokeJsonParser, HttpClient httpClient) {
        this.jokeJsonParser = jokeJsonParser;
        this.httpClient = httpClient;
    }

    Collection<Joke> fetchRandomJokes(int limit) {
        if (limit < 0) {
            throw new IllegalArgumentException("Limit must be a positive integer");
        }
        List<CompletableFuture<Joke>> jokeResponses = sendRandomJokeRequests(limit);
        return joinAll(jokeResponses);
    }

    private List<CompletableFuture<Joke>> sendRandomJokeRequests(int limit) { // List<CompletableFuture<Joke>> result = new ArrayList<>(); // Alternative sendRandomJokeRequest(int limit)
        return IntStream
            .range(0, limit) // for (int i = 0; i < limit; i++) {
            .mapToObj(_ -> sendAsyncRandomJokeRequestWith(httpClient)) // CompletableFuture<Joke> jokeCompletableFuture = sendAsyncRandomJokeRequestWith(httpClient);
            .toList(); // result.add(jokeCompletableFuture);
        // }
        // return result;
    }

    private CompletableFuture<Joke> sendAsyncRandomJokeRequestWith(HttpClient client) {
        return client.sendAsync(request(JOKE_API_URL), HttpResponse.BodyHandlers.ofString())
            .thenApply(HttpResponse::body)
            .thenApply(jokeJsonParser::parse);
    }

    private static HttpRequest request(String url) {
        return HttpRequest.newBuilder()
            .uri(URI.create(url))
            .timeout(Duration.ofSeconds(10))
            .build();
    }

    private static <T> List<T> joinAll(List<CompletableFuture<T>> futures) {
        return futures.stream()
            .map(CompletableFuture::join)
            .toList();
    }
}
```

Schaue dir den Unit-Test **JokeFetcherTest** genauer an! Dieser demonstriert, wie man nebenläufige Programme testen kann.

Zum einen wird Dependency Injection verwendet, um HttpClient in JokeFetcher mit Mockito zu mocken. So ist für den UnitTest keine reale Internetverbindung mehr notwendig. Einer der Testfälle prüft JokeFetcher trotzdem noch mit realen Internetanfragen. Dies soll sicherstellen, dass keine Diskrepanz zwischen gemockten und realen Verhalten vorliegt.

Um die Nebenläufigkeit zu testen, wird ein spezieller HttpClient gemockt, der für eine bestimmte Zeit (100 ms) schläft, bevor er das Ergebnis zurückgibt. Davon sollen 10 parallel ausgeführt werden – die Laufzeit sollte weniger als 1000 ms betragen (im Gegensatz zur seriellen Ausführung). Genau dies prüft einer der Testfälle, um sicherzugehen, dass die Implementierung wirklich nebenläufig ist.